

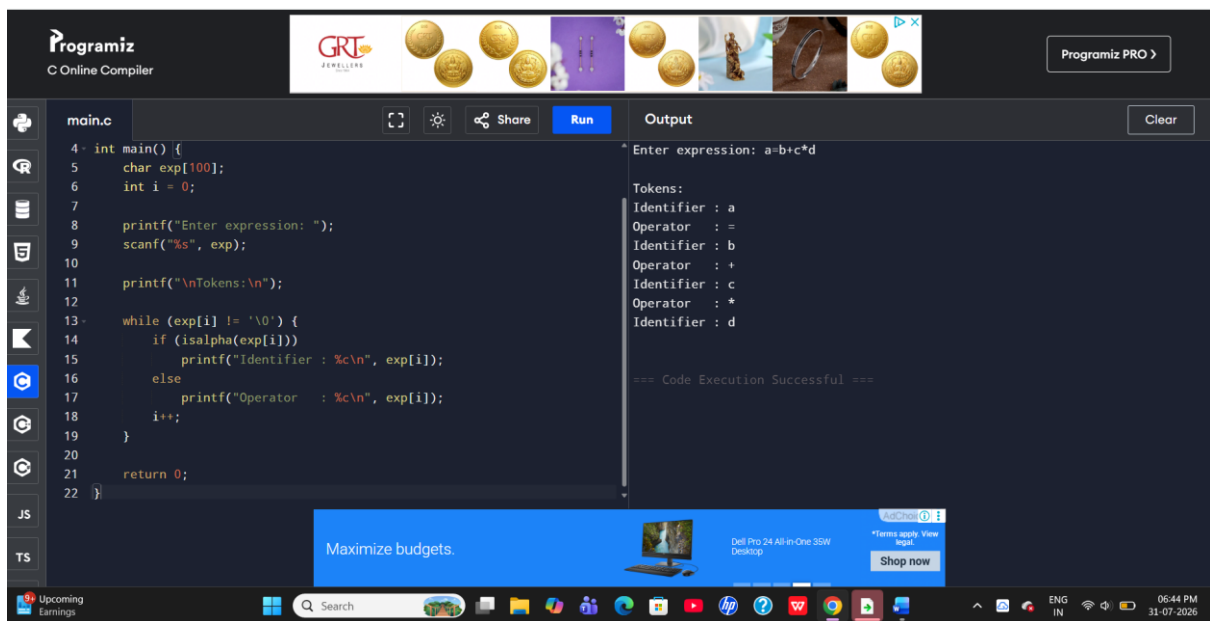
NAME: AKSHAYA A

REG NO:192421222

COURSE: CSA1402 COMPILER DESIGN

CO2_AT1_DESIGN TASK

1. Create a simple lexical analyzer that identifies identifiers and operators in the input expression.



The screenshot displays the Programiz C Online Compiler interface. The code editor on the left contains a C program named `main.c` that implements a simple lexical analyzer. The program prompts the user to enter an expression, reads it, and then iterates through each character. If a character is an identifier (checked using `isalpha`), it is printed as an identifier. If it is an operator (checked using `isop`), it is printed as an operator. The program successfully processes the input expression `a=b+c*d`.

```
4 int main() {
5     char exp[100];
6     int i = 0;
7
8     printf("Enter expression: ");
9     scanf("%s", exp);
10
11    printf("\nTokens:\n");
12
13    while (exp[i] != '\0') {
14        if (isalpha(exp[i]))
15            printf("Identifier : %c\n", exp[i]);
16        else
17            printf("Operator : %c\n", exp[i]);
18        i++;
19    }
20
21    return 0;
22 }
```

The output window on the right shows the results of the program's execution for the input `a=b+c*d`. It lists the tokens as follows:

```
Enter expression: a=b+c*d
Tokens:
Identifier : a
Operator : =
Identifier : b
Operator : +
Identifier : c
Operator : *
Identifier : d
=== Code Execution Successful ===
```

The bottom of the image shows a Windows taskbar with various application icons and a system clock indicating the time is 06:44 PM on 31-07-2026.

2. Propose a simple parsing algorithm and demonstrate parsing of an input expression.

The screenshot shows a C program in a code editor with the following code:

```
1 #include <stdio.h>
2
3 int main() {
4     char input[50];
5     int i = 0;
6
7     printf("Enter expression: ");
8     scanf("%s", input);
9
10    printf("\nParsing Steps:\n");
11
12    while (input[i] != '\0') {
13        printf("Read: %c\n", input[i]);
14        i++;
15    }
16
17    printf("\nParsing Completed Successfully.\n");
18
19    return 0;
20 }
```

The output window shows the execution results for the input expression "id*id+id":

```
Enter expression: id*id+id
Parsing Steps:
Read: i
Read: d
Read: *
Read: i
Read: d
Read: +
Read: i
Read: d
Parsing Completed Successfully.
=== Code Execution Successful ===
```

3. Design a simple hybrid parser that uses LL(1) for simple expressions and LR parsing for complex expressions.

The screenshot shows a C program in a code editor with the following code:

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char exp[50];
6
7     printf("Enter expression: ");
8     scanf("%s", exp);
9
10    if (strchr(exp, '+') || strchr(exp, '/'))
11        printf("Complex expression -> Use LR Parser\n");
12    else
13        printf("Simple expression -> Use LL(1) Parser\n");
14
15    printf("Parsing Successful.\n");
16
17    return 0;
18 }
```

The output window shows the execution results for the input expression "id+id":

```
Enter expression: id+id
Simple expression -> Use LL(1) Parser
Parsing Successful.
=== Code Execution Successful ===
```

4. Critically Evaluate the Effectiveness of Parsing Expression Grammars (PEG)

